A console-first workstation for local models and coding agents

Design note on `cdl-linux`, for readers deciding whether to build it

Jeremy Manning

3 September 2026

The machine is already there, and it is mostly asleep

A laptop with a discrete GPU is a compromise in both directions. It is heavier and hotter than a laptop needs to be, and the GPU sits idle whenever the lid is shut, which is most of the time. A Lambda TensorBook with 64 GB of RAM and an RTX 3080 spends most of its life as an expensive way to read email.

We propose to stop treating it as a laptop: put it in the office, leave it on, reach it over SSH, and let it do three things continuously, namely serve local models to whatever device you are actually using, run coding agents in sessions that survive a dropped connection, and fine-tune models on a GPU that would otherwise be idle. In other words, the hardware is already bought (and mostly unused), so what we are proposing is a change of posture rather than of capability.

A second motivation may matter more than it sounds, which is that configuring a machine this carefully is something we would like to do once rather than repeatedly. The setup is therefore a script rather than a memory of what worked, so that the machine can be rebuilt from it after a failure, a wipe, or a decision to move to different hardware.

What it is

`cdl-linux` is a reproducible workstation configuration for Ubuntu Server 26.04. In practice it is one install script, which runs on a stock machine and turns it into that workstation. The model is Lambda Stack, which is already on this hardware and installs with a single piped command: one line, vanilla Ubuntu, hardware detected, everything configured. We want that experience, and running it twice should change nothing the second time.

There are two supported modes. **Portable** runs the script on an existing Ubuntu Server install and leaves storage exactly as it is; it is the mode that works on anyone else's machine. **Appliance** installs Ubuntu from a supplied autoinstall profile first, to get the storage layout below, and is specific to a two-drive Tensorbook. All of the destructive risk in this project lives in the second mode, and the profile is an optional hardware-specific companion rather than a distribution.

It carries four agent CLIs (Claude Code, Codex, Gemini and OpenCode), Ollama and `llama.cpp` for serving models, a CUDA and PyTorch stack for training, Tailscale and SSH for reach, a small web dashboard, and a console configured to be pleasant rather than default.

It is deliberately not a distribution. A deep audit set out what a real one needs: a remastered installer ISO, signed Debian packages, a signed APT repository with stable and staging channels, signing-key rotation and revocation, release manifests, SBOMs, licence compliance, and an upgrade policy for strangers. That list is right, and every item on it exists to serve people who install your software and then depend on it. We are not making that commitment, so we do not build the apparatus that discharges it. The repository is public and contributions are welcome; guarantees are not offered, and saying so plainly is fairer than implying otherwise by omission.

It is also not an agent orchestration system (agents run in `zellij` workspaces and a human decides what to run), and not multi-user, though that means one supported human operator rather than one Unix account: services still run as their own least-privileged users.

Console-first, which is not the same as headless

There is no graphical stack: no X, no Wayland, no compositor, no display manager, no autologin. But an earlier draft of this design said "headless" and quietly meant two things by it, the second being that the local console was only good enough for emergencies. That was a mistake, and it dropped a requirement recorded early on as high priority rather than cosmetic.

It matters concretely, because the encrypted root is unlocked by a passphrase typed at the keyboard. There is deliberately no remote unlock, so any reboot means someone stands at the machine, and the screen they are looking at should be somewhere work happens. Using a local model from the console is a first-class use case rather than a fallback.

Ligatures on a text console are a genuine technical obstacle, and the way around it is specific enough to state. The kernel's virtual terminal draws 1-bit bitmap glyphs from a 512-entry table and has no text-shaping engine at all, while a ligature is a shaping substitution, so no console font can produce one. We therefore run `kmscon` on the first terminal, which renders through freetype and does shape, giving Fira Code with its ligatures on the machine's own screen without a compositor and without the screen-locking problem a compositor would bring. The second terminal stays an ordinary kernel console, deliberately: `kmscon` is one more thing that can fail, and the recovery terminal must not depend on it.

How the rest works

Storage. Only in appliance mode. The two 1 TB NVMe drives are striped into one volume: RAID0, LUKS on top, then btrfs with subvolumes for the system, home and model weights. `/boot` sits outside the encryption because GRUB has to read a kernel before anything can be unlocked. Striping is a deliberate trade (we recommended against it and were overruled for defensible reasons), and the spec records both sides rather than only the conclusion.

Serving models. Ollama is the endpoint, on a fixed port bound to the Tailscale network, and it is what other devices point at. `llama.cpp` behind `llama-swap` is a second, separate endpoint on localhost for the cases Ollama does not cover. There is no reverse proxy and no unified router, because inventing one is work with no payoff at this scale.

Training. A wrapper takes an exclusive lock on the GPU, stops the model servers, runs, then restarts them. Serving and training do not overlap. On 16 GB of VRAM it is the one policy of the three we considered whose behaviour is predictable, and it is honest about the cost: a request to the model endpoint during a training run is refused rather than answered slowly.

Backups go to a Hugging Face bucket through `restic`, with a second copy pulled by a different machine that holds no credential this one can read.

What to push on before we build this

Ordered by how much damage a wrong answer does. We would rather hear about these now.

1. Right now the machine has no redundancy and no protected backup, at the same time. RAID0 means either drive failing destroys the volume (both drives, not one). Separately, the Hugging Face bucket has no versioning and no lifecycle rules, so a write-capable token on the box can erase the backup history permanently and irrecoverably. Our answer is a second copy, pulled by a machine for which the box holds no credential. We think that is sound, and it is also (at present) the only thing standing between one bad command and total loss. What would settle it: build the second copy first, before anything else, and test recovering from it.

2. Every reboot needs someone at the machine. The encrypted root is unlocked at the console and there is deliberately no remote unlock, so a power cut on a Friday leaves an unreachable machine until someone drives in (and from outside, an unreachable machine looks identical to a dead one). A UPS would convert the commonest cause into nothing at all, and may be the cheapest available fix. Binding the key to the TPM removes the trip, but means anyone who steals the whole machine and boots it gets the data.

3. Part of the storage layout cannot be built by the stock installer, and the fix is riskier than it first looked. A VM install settled the easy part: RAID0, LUKS, btrfs, `/boot` outside the encryption and an unlock at the console all work, and the machine boots off the array. What the installer will not do is create subvolumes; it puts root in the filesystem's top level, which costs the ability to snapshot the system independently of home, and that was the reason for having subvolumes at all.

An earlier version of this document said the answer was to migrate root into a subvolume from the install script afterwards. **That turns out to be impossible**, and finding out took two VM cycles: btrfs will not snapshot the top-level subvolume while it is mounted as root, and a directory with a filesystem mounted on it cannot be renamed, which rules out both obvious methods. The migration now runs during installation, where the filesystem is mounted but nothing is executing from it.

Three further attempts failed for reasons none of us would have predicted from the documentation: `mv` across a btrfs subvolume is a copy rather than a rename, so it silently breaks hardlinks; cloud-init's `write_files` does not reach the installer environment where the script has to run; and `/run` is mounted `noexec` there, so a delivered script cannot be executed however its permissions are set. Each cost a VM cycle. **On the Tensorbook, the first would have cost a reinstall and the second would have left a half-migrated root.**

The honest status is that this path is experimental, and it is gated on running twice from clean disks, with a fixture of hidden files, hardlinks, symlinks, awkward filenames and multiple owners that has to survive intact, before it goes near a machine with work on it.

4. Secure Boot here means signed kernel and modules, not verified boot. Ubuntu's chain validates the shim, GRUB, the kernel and its modules, but the initrd is not validated and `/boot` is unencrypted by necessity. Anyone with physical access can alter what runs before the disk is unlocked. We state this narrowly on purpose, since a security property described more broadly than it holds is worse than one nobody claimed.

5. The GPU lock is cooperative, and cooperative locks are honoured only by those who remember them. Anything that runs a training script directly, without the wrapper, takes VRAM and the lock stops nothing. Making the wrapper the path of least resistance is the only enforcement available short of containers, which do not earn their cost here.

6. Being on the tailnet is not the same as being authorised. The dashboard and the model endpoint are reachable by every device on the tailnet, and by every device shared into it, which means prompts and output too. The dashboard checks caller identity; the model endpoint cannot, because Ollama has no per-caller authentication. For a one-person tailnet that is close enough to fine, and if the tailnet is ever shared, the endpoint is shared with it.

7. Finally, the premise deserves a challenge. Building this makes sense if a GPU you can reach from anywhere, all the time, does more for you than a GPU in a laptop you open sometimes. If the answer turns out to be that the models you actually use are hosted ones (and that the local GPU is a hobby), then this may be a weekend of setup in service of a workflow that does not yet exist. What would settle it: build the first three milestones, use the machine over SSH for a week, and notice whether you keep going back to it.

Where this stands

The design is specified and none of it is installed on the Tensorbook yet. Several of its riskiest questions have been answered by experiment rather than by argument, and two of those experiments changed the design.

The backup path was one. Testing it showed that `restic` cannot initialise a repository on a Hugging Face bucket through its own S3 support at all, because it reads the account namespace as the bucket name. Routed through `rclone` instead, every step passes, including a byte-identical restore. That added a dependency the design did not previously have.

The storage layout was the other. The VM harness found that the installer will not build the layout the way the spec described it, and, separately, that the harness's own first verifier would have passed a filesystem that did not match. Both are better found in a virtual machine than on a laptop with your work on it, and the second is the more uncomfortable of the two: a test that passes the wrong answer is worse than no test, because it converts an open question into a false one.

What we would like from a reader: a judgement on the premise, an opinion on whether striping two drives with no immutable backup is a trade you would make, and any failure mode in the boot or recovery path we have not thought of.